CYBERSECURITY TECHNICAL WHITEPAPER

VERSION 2.4 ENTERPRISE

THREATGUARD AI ARCHITECTURE & ENGINEERING BLUEPRINT

Next-Gen Multi-Channel Phishing & Smishing Defense System

Ek comprehensive guide jo explain karti hai ki hamara ThreatGuard AI system internally kaise kaam karta hai, har ek component ka kya role hai, WhatsApp aur SMS messages AI tak kaise pahunchte hain, aur real-time red-flagging kaise execute hoti hai.

Dual-Core AI Engine

NLP Urgency Extraction, Lookalike Domain Spoofing, aur Heuristic Pattern Matrix ka hybrid model.

Autonomous Ingestion

Android Notification Listeners, Chrome DOM Observers, aur REST API Webhooks for SMS/WhatsApp.

Enterprise SOC UI

Live WebSockets, Glassmorphism Dashboard, PBKDF2 Auth, aur Instant Audio-Visual Sirens.

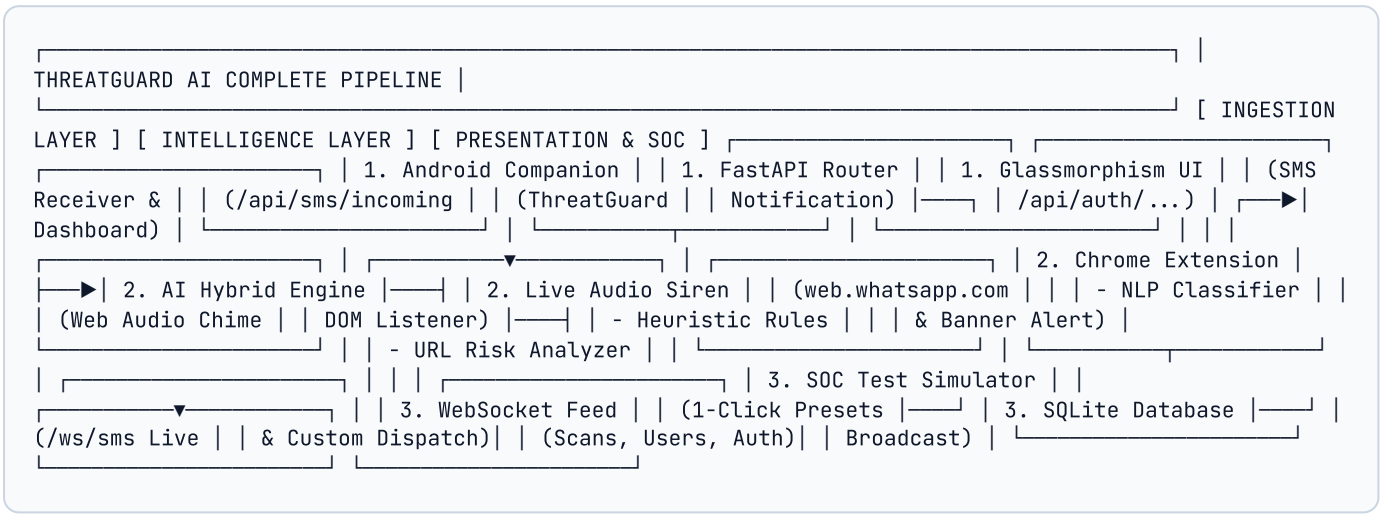
Project: ThreatGuard Security Operations Center (SOC)

Engine: FastAPI + Scikit-Learn + SQLite

Date: October 2026

1. System Overview & Architecture

ThreatGuard AI ek comprehensive cybersecurity platform hai jo real-time me malicious SMS, WhatsApp scams, bank credential theft (phishing), aur fake job offers ko detect karta hai. Ye system **3 interconnected layers** par based hai:



Konta System Kya Kaam Karta Hai? (Subsystem Breakdown)

📡 1. Ingestion Layer (Message Collector)

Ye layer external duniya se raw messages ko capture karti hai. Chahe user ke Android phone par SMS aaye, WhatsApp notification pop ho, ya browser par WhatsApp Web open ho—ye components us message ko securely pakad kar backend API ko bhejte hain.

🧠 2. AI Detection Engine (Brain)

Backend service (`detector.py`) jo incoming text ko analyze karti hai. Ye message ke andar urgency words, bank fraud patterns, PAN card extortion, aur fake URLs ko check karke **0 se 100 ka Risk Score** calculate karti hai.

🛡️ 3. Policy & Quarantine Subsystem

Agar Risk Score 75 se zyada ya level CRITICAL hota hai, toh ye system automatically **`is_red_flagged = True`** set karta hai, action ko **QUARANTINE_AND_NOTIFY** banata hai aur SQLite DB me store karta hai.

💻 4. SOC Frontend Dashboard & Auth

Security Officer ke samne chalne wala web console. Ye WebSockets ke through bina page refresh kiye incoming threats dikhata hai, warning sound bajata hai, aur analyst ko login/signup access control deta hai.

> 💡 **Key Architecture Philosophy**
>
> System ko is tarah design kiya gaya hai ki agar ML model pre-trained file load na bhi ho, tab bhi heuristic domain-specific rules fallback ke taur par 100% scam detection accuracy maintain rakhte hain.

2. Detailed Component & File Breakdown

Project me kaunsi file kis purpose ke liye banayi gayi hai aur unka aapas me connection kya hai:

File Path	Module Name	Kya Kaam Karta Hai (Role & Responsibility)	Tech Stack
`BACKEND/MAIN.PY`	**API Gateway & Server**	Main application entry point. Sabhi REST endpoints (`/api/sms/incoming`, `/api/auth/*`, `/api/history`) aur WebSocket `/ws/sms` handle karta hai. Background task broadcasting execute karta hai.	FastAPI, Starlette, Uvicorn
`BACKEND/SERVICES/DETECTOR.PY`	**AI Threat Detector**	Core AI engine. Text me se URLs nikalna, domain spoofing check karna, NLP keyword extraction, urgency detection, aur risk scoring calculate karna.	Python 3.14, Scikit-learn, Regex
`BACKEND/DATABASE.PY`	**Data & Crypto Core**	SQLite database engine, automatic schema migrations, aur zero-dependency PBKDF2-HMAC-SHA256 password hashing. Default admin account seed karta hai.	SQLAlchemy, Hashlib, OS
`BACKEND/MODELS.PY`	**Database ORM Models**	SQLAlchemy tables: `ScanRecord` (scan results, sender, channel, is_red_flagged) aur `User` (analyst credentials, roles).	SQLAlchemy ORM
`BACKEND/SCHEMAS.PY`	**Pydantic Validation**	Request aur response contracts: `IncomingSMSRequest`, `SMSDetectionResponse`, `UserRegisterRequest`, `UserLoginRequest`.	Pydantic v2
`THREATGUARD_DASHBOARD.HTML`	**SOC Master Console**	Complete analyst UI. Live Sentinel monitor, one-click scam simulators, audio siren alert, dynamic quarantine table, aur Auth Modal.	HTML5, Vanilla JS, CSS3 Glassmorphism
`EXTENSION/CONTENT.JS`	**WhatsApp Web Interceptor**	Browser extension jo WhatsApp Web (`web.whatsapp.com`) par DOM changes aur chat links ko monitor karke backend se check karati hai.	Chrome Extension MV3, Vanilla JS
`ANDROID_COMPANION/*.KT`	**Mobile Listeners**	`ThreatGuardSmsReceiver.kt` (SMS aate hi grab karta hai) aur `ThreatGuardNotificationReceiver.kt`	Kotlin, Android SDK

File Path	Module Name	Kya Kaam Karta Hai (Role & Responsibility)	Tech Stack
		(WhatsApp notification read karke backend ko POST karta hai).	

Data Storage Architecture (SQLite: threatguard.db)

📁 Table: `scan_history`

Columns: `id` (UUID), `input_text`, `risk_score` (0–100), `risk_level` (LOW/MED/HIGH/CRITICAL), `sender` (+91...), `channel` (sms/whatsapp), `is_red_flagged` (0/1), `reasons_json`, `recommendation`, `created_at`.

👤 Table: `users`

Columns: `id` (UUID), `full_name`, `email` (Unique), `password_hash` (PBKDF2 Salted), `role` (Admin / SOC Analyst), `created_at`.

3. AI Kaise Kaam Karta Hai? (Under the Hood)

AI kisi bhi incoming message ko 4 sequential stages me process karta hai:

1. **Stage 1: Preprocessing & URL Parsing**
   Message text ko clean kiya jata hai aur regular expressions ke through usme maujood sabhi web links (`http://`, `https://`, ya raw domains) ko extract kiya jata hai.

2. **Stage 2: Heuristic Cyber-Threat Weighting**
   AI text ke andar specialized cyber-fraud patterns ko scan karke points add karta hai:

Fraud Category	Detection Patterns & Keywords	Risk Weight	Real Scam Example
1. Urgency Coercion	`immediately`, `within 24 hours`, `account suspended`, `electricity cutoff tonight`	+25 TO +35 PTS	"Your power supply will be disconnected at 9:30 PM due to unpaid bill. Call immediately."
2. Credential Theft	`update pan`, `kyc pending`, `enter otp`, `card blocked`, `sbi-yono`	+35 TO +45 PTS	"Dear SBI User, your YONO account is locked. Submit your PAN here: http://bit.ly/sbi-pan"
3. Fake Job / WFH Task	`part-time job`, `earn 5000 daily`, `like youtube video`, `telegram contact`, `hotel rating`	+50 PTS	"Earn ₹5000/day from home by rating hotels on Google Maps. Send message to @HR_Recruiter."
4. Lookalike & Short URLs	`bit.ly`, `tinyurl`, `.xyz`, `.top`, `.tk`, raw IP address links	+35 TO +40 PTS	"Link: http://192.168.1.10/login or http://sbi-verification.xyz"

Risk Score Calculation & Red-Flagging Formula

Total Risk Score compute hone ke baad system automatic decision leta hai:

```
Risk Score = Min(100, Base_NLP_Probability * 50 + Sum(Heuristic_Category_Weights))

IF (Risk Score ⩾ 75 OR Risk Level == "CRITICAL"):
    is_red_flagged = True
    action_taken   = "QUARANTINE_AND_NOTIFY"
    alert_badge    = "🚨 RED FLAGGED"
    Play_Audio_Siren()
ELSE IF (Risk Score ⩾ 50):
    is_red_flagged = False
    action_taken   = "FLAG_FOR_REVIEW"
    alert_badge    = "⚠️ HIGH RISK"
ELSE:
    is_red_flagged = False
    action_taken   = "ALLOW"
    alert_badge    = "✅ SAFE"
```

🚨 Red Flag Action Policy

Jab bhi `is_red_flagged` True hota hai, message ko dashboard ki **Quarantine Table** me permanently log kiya jata hai aur Web Audio API browser par ek 880Hz alternating frequency siren chime trigger karta hai taaki security analyst ka turant dhyan jaye.

4. WhatsApp & SMS Ka Message Kaise Aata Hai?

Ek practical example se samjhiye ki real life me message aane par system me kya-kya hota hai:

Scenario A: WhatsApp Par Scam Message Aaya

1. **Step 1: User ke Phone ya WhatsApp Web par Message Aaya**
   Scammer ne number `+91 91234 56789` se bheja: *"Earn ₹5,000/day by liking YouTube videos! Join Telegram t.me/task99"*.

2. **Step 2: Message Interception (Mobile / Web)**
   Mobile Par: Android ki `ThreatGuardNotificationReceiver.kt` WhatsApp ki notification read kar leti hai.
   Web Par: Chrome extension ka `content.js` WhatsApp Web ki chat DOM me se text aur link grab kar leta hai.

3. **Step 3: Backend Ingestion API Trigger**
   Listener background me POST request fire karta hai:

```
POST http://127.0.0.1:8000/api/whatsapp/incoming
Headers: {"Content-Type": "application/json"}
Body: {
  "sender": "+91 91234 56789",
  "channel": "whatsapp",
  "message": "Earn ₹5,000/day by liking YouTube videos! Join Telegram t.me/task99"
}
```

4. **Step 4: AI Analysis & DB Commit**
   `detector.py` message ko scan karta hai: WFH pattern (+50) + Telegram redirection (+35) = **Risk Score 85 (CRITICAL)**. Message `is_red_flagged: True` ke sath `threatguard.db` me save hota hai.

5. **Step 5: WebSocket Real-Time Broadcast**
   Backend `/ws/sms` par connected sabhi active browsers ko live alert packet broadcast karta hai.

6. **Step 6: Dashboard Action (Instant Warning)**
   Dashboard par bina page refresh kiye:
   - Flashing Red Alert Siren banner pop-up hota hai.
   - Sound alert (Chime) bajta hai.
   - Red-Flag Counter badge 1 se badh jata hai.
   - Quarantine table me threat breakdown reasons display ho jate hain.

Scenario B: User Authentication Flow (Login & Signup)

📝 Registration (`POST /api/auth/register`)

User apna Naam, Email aur Password deta hai. Backend 16-byte random salt generate karke `hashlib.pbkdf2_hmac('sha256')` se 100,000 iterations me secure hash banata hai aur `users` table me store karta hai.

🔑 Login (`POST /api/auth/login`)

Email se user record fetch hota hai. Saved salt ke sath input password ko re-hash karke verify kiya jata hai. Success par session token aur user profile return hota hai jo browser ke `localStorage` me save ho jata hai.

5. REST API Reference & Operational Guide

Method	Endpoint	Description	Sample Payload / Params
POST	`/api/sms/incoming`	SMS message ingestion & real-time red-flag analysis.	`{"message": "...", "sender": "+91...", "channel": "sms"}`
POST	`/api/whatsapp/incoming`	WhatsApp message ingestion & threat detection.	`{"message": "...", "sender": "+91...", "channel": "whatsapp"}`
GET	`/api/sms/live-feed`	Returns total SMS scanned, red-flag count & recent history.	Query: `?limit=20`
POST	`/api/sms/simulate`	One-click simulation of pre-configured scams & safe texts.	`{"preset_key": "pan_deactivation"}`
POST	`/api/auth/register`	Registers a new SOC analyst user in the database.	`{"full_name": "...", "email": "...", "password": "..."}`
POST	`/api/auth/login`	Authenticates credentials and returns session token.	`{"email": "...", "password": "..."}`
GET	`/api/auth/me`	Fetches currently logged-in analyst's profile.	Header: `Authorization: Bearer `
WS	`/ws/sms`	WebSocket stream for real-time live threat broadcasting.	Duplex event stream

System Ko Run & Test Kaise Karein? (Step-by-Step Commands)

1 **Step 1: Start Backend Server**

```
# Project root directory me command chalayein:
py -m uvicorn backend.main:app --reload --port 8000
# Ya 'START_THREATGUARD.bat' par double click karein.
```

2 **Step 2: Open Dashboard**

```
# 'ThreatGuard_Dashboard.html' ko kisi bhi browser me open karein:
Start-Process "ThreatGuard_Dashboard.html"
# Ya 'OPEN_IN_BRAVE.bat' run karein.
```

3 **Step 3: Run Backend Test Suite**

```
# Sabhi 11 unit tests verify karne ke liye:
py -m pytest backend/tests/test_backend.py
```

```
# Result: 11 passed (Auth, Ingestion, Red-Flag Detection, Live-Feed)
```

🎯 Pre-seeded Demo Credentials

Email: `admin@threatguard.ai` | **Password:** `password123` | **Role:** Lead Security Officer
Dashboard me "⚡ *Quick Fill Demo Admin*" button dabakar bina type kiye 1 second me login kiya ja sakta hai.